

Formal Methods and Automata Theory

1. Alphabets, Strings, and Languages

To understand automata theory, you must first define the fundamental units of formal languages.

Alphabets

An alphabet is a finite, nonempty set of symbols. You will normally see an alphabet denoted by the Greek letter Σ (Sigma). When discussing multiple alphabets, you can use subscripts (e.g., Σ_1, Σ_2) or other symbols like G .

- Binary alphabet: $\Sigma = \{0, 1\}$
- Letter alphabet: $\Sigma = \{a, b, c\}$
- Mixed alphabet: $\Sigma = \{a, b, c, \&, z\}$
- Symbol alphabet: $\Sigma = \{\#, \nabla, \spadesuit, \beta\}$

Strings or Words

A string (or word) over an alphabet Σ is a finite sequence of concatenated symbols from Σ .

- For the binary alphabet $\{0, 1\}$, valid strings include 0110, 11, and 001.
- For the alphabet $\{a, b, c\}$, valid strings include aab , $abcb$, b , and cc .

A string does not need to contain all the symbols from its alphabet. The string cc over $\{a, b, c\}$ omits a and b . Because of this, a string over an alphabet is automatically a string over any superset of that alphabet.

Length of a String

The number of symbols in a string w is its length, denoted by $|w|$.

- $|011| = 3$
- $|11| = 2$
- $|b| = 1$
- $|1001101| = 7$
- The empty string is denoted by ϵ . $|\epsilon| = 0$.

For any symbol c and string s , you can use the function $\#_c(s)$ to count the number of times c occurs in s . For example, $\#_a(abbaaa) = 4$.

In software, a basic string function like `length("helloworld")` returns 11. It evaluates the raw character count without counting terminator characters or modifying the string itself.

String Operations

You can manipulate strings using several specific operations.

Concatenation: Joining two strings together. If you concatenate x and y , denoted by xy , you place a copy of x followed immediately by a copy of y without any intervening space.

String Parts: Consider the string 011 over the binary alphabet.

- **Prefixes:** $\epsilon, 0, 01, 011$
- **Suffixes:** $\epsilon, 1, 11, 011$
- **Substrings:** $\epsilon, 0, 1, 01, 11, 011$

Every string x is a prefix, suffix, and substring of itself. The empty string ϵ is a prefix, suffix, and substring of every string.

A string x is a **proper** prefix, suffix, or substring if it is not exactly equal to the original string. In the example above, all prefixes except 011 are proper prefixes.

Relations on Strings: A string s is a substring of t if s occurs contiguously within t . For example, aaa is a substring of $aaabbbbaaa$, but $aaaaaa$ is not. A string s is a proper substring of t if s is a substring of t and $s \neq t$.

Powers of Strings: For any string x and integer $n \geq 0$, the notation x^n denotes the string formed by concatenating n copies of x . You define this inductively:

- $x^0 = \epsilon$
- $x^n = xx^{n-1}$

If $x = 011$, then $x^3 = 011011011$, $x^1 = 011$, and $x^0 = \epsilon$.

Reversal: For any string $w_n = a_1a_2a_3 \dots a_n$, the reversal is $w = a_n \dots a_3a_2a_1$. If you reverse a concatenated string, $(wx) = xw$. For example, $(namta) = (ta)(nam) = atman$.

Powers of Alphabets

You write Σ^k to denote the set of strings of length k using symbols from Σ . The set Σ^0 always equals $\{\epsilon\}$.

Powers of Σ : $\Sigma = \{0,1\}$

Σ^0 = Set of all strings of length 0 : $\Sigma^0 = \{\epsilon\}$

Σ^1 = Set of all strings of length 1 : $\Sigma^1 = \{0,1\}$

Σ^2 = Set of all strings of length 2 : $\Sigma^2 = \{00, 01, 10, 11\}$

Σ^3 = Set of all strings of length 3 : $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

For the binary alphabet $\{0, 1\}$:

- $\Sigma^0 = \{\epsilon\}$
- $\Sigma^1 = \{0, 1\}$
- $\Sigma^2 = \{00, 01, 10, 11\}$
- $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

The set of all strings over Σ is denoted by Σ^* . This is the union of all powers:

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \cup \Sigma^n$. It contains every string generated by iteratively concatenating symbols any number of times.

If $\Sigma = \{a, b\}$, then $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$.

If Σ is the empty set, then $\Sigma^* = \{\epsilon\}$. You can proceed from an empty set to a non-empty set containing the empty string via iterated concatenation.

Languages

A language over an alphabet is any subset of Σ^* .

Examples of languages include:

1. The empty language.
2. The entire set Σ^* .
3. The language $\{\epsilon\}$. Note that $\{\epsilon\}$ because the empty set contains zero strings, while $\{\epsilon\}$ contains one string of length zero.
4. The set of all strings over $\{0, 1\}$ containing an equal number of 0s and 1s.
5. The set of all strings over $\{a, b, c\}$ starting with a .

You normally use capital letters like A, B, C, or L to denote languages.

Set Operations on Languages: You can apply standard set mathematics to languages.

- **Union ($L_1 \cup L_2$):** A string $x \in L_1 \cup L_2$ if $x \in L_1$ or $x \in L_2$.
 - Example: $\{0, 11, 01, 011\} \cup \{1, 01, 110\} = \{0, 1, 11, 01, 011, 110\}$.
- **Intersection ($L_1 \cap L_2$):** A string $x \in L_1 \cap L_2$ if $x \in L_1$ and $x \in L_2$.
 - Example: $\{0, 11, 01, 011\} \cap \{1, 01, 110\} = \{01\}$.
- **Complement:** The complement of L is taken with respect to the universe Σ . The complement is $\bar{L} = \{x \in \Sigma \mid x \notin L\}$.
 - Example: If $L = \{x \mid |x| \text{ is even}\}$, the complement is $\{x \mid |x| \text{ is odd}\}$.
- **Reversal (L^R):** $L^R = \{w^R \mid w \in L\}$.
 - Example: If $L = \{0, 11, 01, 011\}$, then $L^R = \{0, 11, 10, 110\}$.

Language Concatenation: The concatenation of L_1 and L_2 is $L_1 L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

- Example: $\{a, ab\} \{b, ba\} = \{ab, aba, abb, abba\}$.
- Properties: $L_1 L_2 \neq L_2 L_1$ in general. $L \cdot \epsilon = \epsilon \cdot L = L$.

Iterated Concatenation: You denote the concatenation of L with itself n times as L^n .

- $L^0 = \{\epsilon\}$
- $L^n = L L^{n-1}$

Example: If $L = \{a, ab\}$:

- $L^0 = \{\epsilon\}$
- $L^1 = L \{\epsilon\} = \{a, ab\}$
- $L^2 = L L^1 = \{aa, aab, aba, abab\}$
- $L^3 = L L^2 = \{aaa, aaab, aaba, aabab, abaa, abaab, ababa, ababab\}$

Finite State Machine (Prerequisites)

Symbol - $a, b, c, 0, 1, 2, 3, \dots$

Alphabet - Σ - collection of symbols - Eg. $\{a, b\}, \{d, e, f, g\}$

String - sequence of symbols. Eg. $\{0, 1, 2\}$

Language - set of strings
Eg. $\Sigma = \{0, 1\}$
 $a, b, 0, 1, aa, bb, ab, 01, \dots$

$L_1 =$ set of all strings of length 2
 $= \{00, 01, 10, 11\}$



2. Automata Theory Fundamentals

Automata theory is the study of abstract mathematical machines or systems and the computational problems you can solve using them. These abstract machines are called automata.

An automaton uses states (drawn as circles) and transitions (drawn as arrows). When the automaton reads an input symbol, it makes a transition to another state according to its transition function.

This field closely relates to formal language theory. Automata classify formal languages by recognizing them. An automaton serves as a finite representation of a formal language, even if that language contains an infinite number of strings.

Automata theory studies several properties:

1. **Recognizable languages:** Which class of formal languages can a specific automaton recognize?
2. **Closure properties:** Are certain automata closed under union, intersection, or complementation?
3. **Language Hierarchy:** How expressive is a type of automata compared to others?
4. **Emptiness checking:** Does a given machine accept any input word at all?
5. **Determinization:** Can you transform a non-deterministic machine into a deterministic one without changing the recognized language?
6. **Minimization:** What is the smallest automaton that recognizes a specific language?

Applications of Automata Theory

Different automata models play varied roles across practical fields.

- **Finite Automata:** Used in text processing, compilers, and hardware design.
- **Context-Free Grammars (CFG):** Used heavily in programming languages, artificial intelligence, and the structural study of human languages.
- **Cellular Automata:** Used in biology to model John Conway's Game of Life, mollusk pigmentation, and pinecone growth patterns. Stephen Wolfram claims machines with finite states and rules could explain the entire universe. He relates automata theory to fluid flow, snowflake formation, crystal formation, chaos theory, cosmology, and financial analysis.
- **Parsing:** Automata run as algorithms to automatically verify if strings follow grammar rules during compiler design.

Basic Automaton Components

Every automaton fulfils basic requirements similar to real computers.

1. **Input Tape:** A mechanism for reading a sequence of symbols from an alphabet. Simple machines read left to right without changing the tape. Powerful versions can read in both directions and change the input symbols.
2. **Output:** An acceptor produces binary output (accept/reject). A transducer produces an output sequence in response to an input sequence.
3. **Temporary Storage:** Optional cells capable of holding symbols. The machine can read and write to this storage. Access capabilities depend on the exact storage type.
4. **Control Unit:** The core feature that exists in one of a finite number of internal states. It changes states via a defined transition function.

Classes of Automata and Languages

You map automata classes to the languages they can recognize.

- **Deterministic Finite Automata (DFA), NFA, and ϵ -NFA:** Recognize Regular Languages.
- **Pushdown Automata (PDA):** Recognize Context-Free Languages.
- **Linear Bounded Automata (LBA):** Recognize Context-Sensitive Languages.
- **Turing Machines:** Recognize Recursively Enumerable Languages.

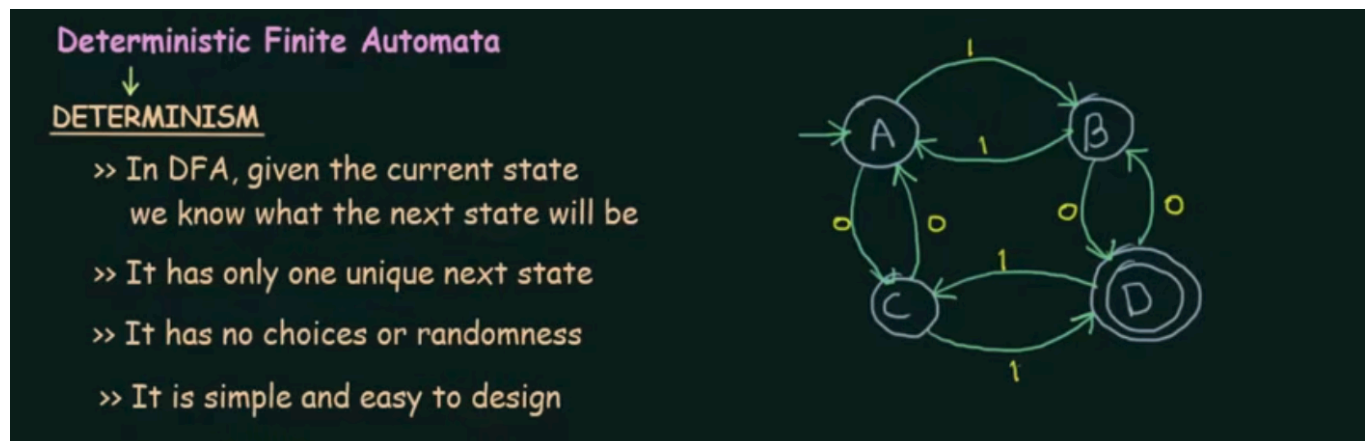
3. Finite State Automata (FSA)

A finite automaton acts as a simple model of computation, originally proposed to model neuron behavior. An FSA has zero temporary storage compared to a PDA or Turing machine. It cannot rewrite input into memory. The control unit retains an explicit but strictly bounded amount of information based purely on its current state.

A state represents an instantaneous description of the system holding all relevant information to determine how the system will evolve. Transitions are instantaneous changes of state reacting to inputs. Digital systems and vending machines are practical examples of finite-state transition systems.

Deterministic Finite Automata (DFA)

A DFA operates without any element of choice. It has a finite number of states and arcs, and it acts as an acceptor to produce a yes/no answer.



A DFA is defined precisely by five parts:

1. Q : A finite set of states.
2. Σ : A finite alphabet of input symbols.
3. q_0 : The initial start state (q_0).
4. F : A set of accepting or final states (F).
5. δ : A transition function mapping a state and symbol to a new state ($\delta : Q \times \Sigma \rightarrow Q$).

You can extend the transition function inductively to process entire strings:

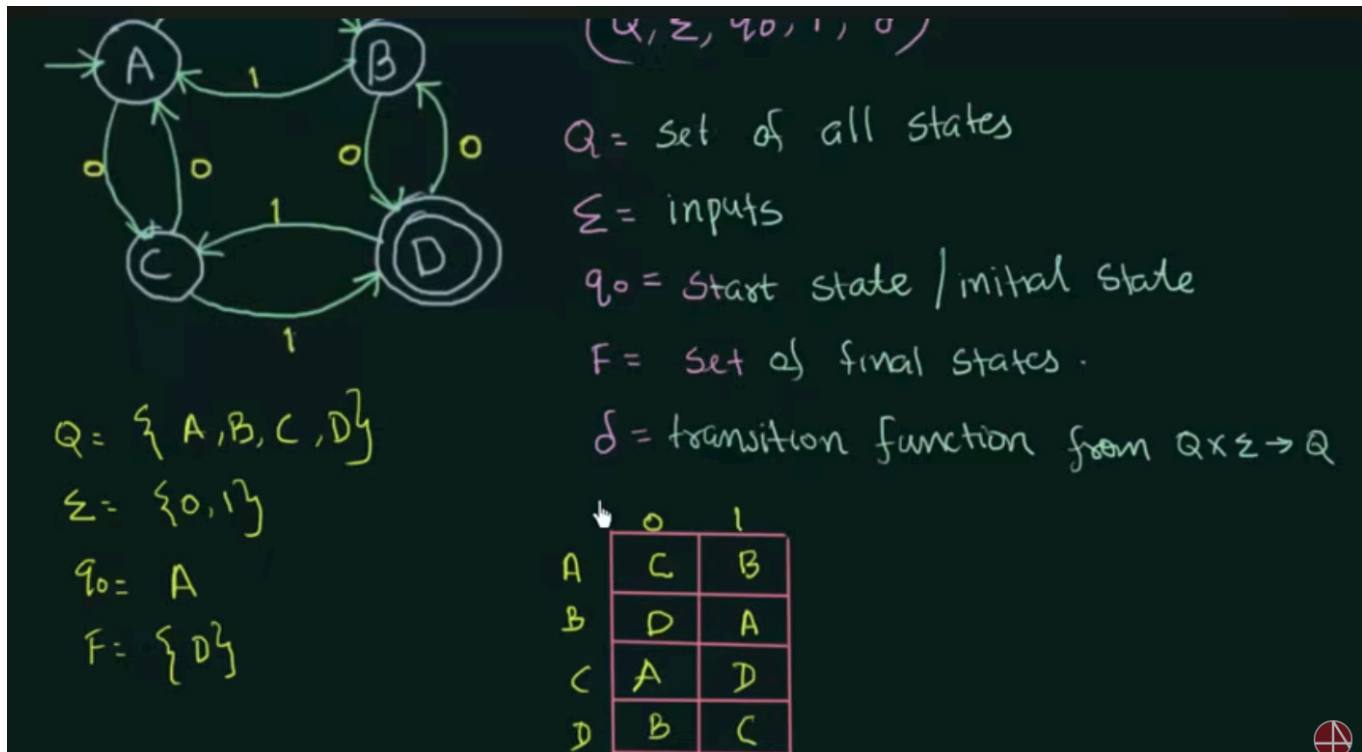
- $\delta(q, \epsilon) = q$
- $\delta(q, ax) = (\delta(q, a), x)$ A machine accepts a string x if the final state $\delta(q_0, x)$ belongs to F .

Algorithm for DFA Operation:

1. Set the "current state" to the start state. Place the "read head" at the beginning of the input string.
2. While characters remain in the string:
 - Read the next character and advance the read head.
 - From the current state, follow the arc labeled with that character. The destination becomes the new current state.

- When you read all characters, accept the string if the current state is a final state. Otherwise, reject it.

Example DFA Trace:



Consider a string 1 0 0 1 1 1 0 0 running through a DFA. A sample trace follows this path:

$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_3 \xrightarrow{0} q_1 \xrightarrow{1} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0 \xrightarrow{0} q_2 \xrightarrow{0} q_0$.

Since the machine ends in q_0 and q_0 is a final state, it accepts the string.

Implementing a DFA in Code:

You can implement a DFA using GOTO statements directly mapping states to labels.

```

q0 : read char;
    if eof then accept string;
    if char = 0 then go to q2;
    if char = 1 then go to q1;
q1 : read char;
    if eof then reject string;
    if char = 0 then go to q3;
    if char = 1 then go to q0;
q2 : read char;
    if eof then reject string;
    if char = 0 then go to q0;
    if char = 1 then go to q3;
q3 : read char;

```



```
if eof then reject string;
if char = 0 then go to q1;
if char = 1 then go to q2;
```

Alternatively, you can use a LOOP and CASE statement.

```
state := q0;
loop
  case state of
    q0 : read char;
        if eof then accept string;
        if char = 0 then state := q2;
        if char = 1 then state := q1;
    q1 : read char;
        if eof then reject string;
        if char = 0 then state := q3;
        if char = 1 then state := q0;
    q2 : read char;
        if eof then reject string;
        if char = 0 then state := q0;
        if char = 1 then state := q3;
    q3 : read char;
        if eof then reject string;
        if char = 0 then state := q1;
        if char = 1 then state := q2;
  end case;
end loop;
```

Ada Identifier Example:

In the Ada programming language, valid identifiers consist of a letter followed by any number of letters, digits, and underlines. The identifier cannot end in an underline or have two underlines in a row. You can build a specific DFA acceptor to recognize exactly this pattern.

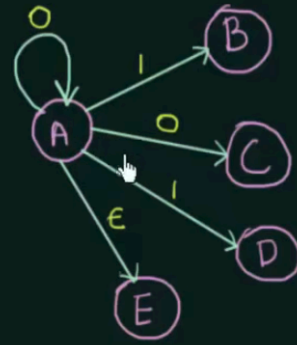
Nondeterministic Finite Automata (NFA)

An NFA makes choices. The transition function maps state and input pairs into sets of possible states. The machine traverses all possible paths in parallel. It allows ϵ transitions (empty string transitions), letting the machine change states without consuming any input character.

Non-deterministic Finite Automata

↓ NON-DETERMINISM

- >> In NFA, given the current state there could be multiple next states
- >> The next state may be chosen at random
- >> All the next states may be chosen in parallel

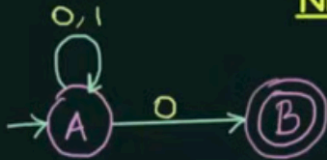


An NFA accepts a string if at least one parallel transition sequence leads to a final state. Multiple transitions on the same character, or zero transitions entirely, are allowed. This looseness makes constructing machines easier, though it does not add computational power over a DFA.

NFA Example Trace:

If an NFA receives the input 010 and starts at q_0 , it might process the first 0 and transition to q_1 , while also taking an ϵ transition to q_3 . Thus, $(q_0, 0) = \{q_1, q_3\}$. It processes 1 to move to q_0 and q_2 . Finally, it processes the last 0 , ending in states q_1, q_2 , or q_3 . Because at least one of these is an accept state, it accepts the string. You can also view this as inserting ϵ symbols into the string to find a valid path: $0\epsilon10$ gives a path $q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_2 \rightarrow q_2$.

NFA - Formal Definition



$L = \{ \text{Set of all strings that end with 0} \}$

$$(Q, \Sigma, q_0, F, \delta)$$

Q = Set of all states

Σ = inputs

q_0 = start state / initial state

F = Set of final states

$$\delta = Q \times \Sigma \rightarrow \underline{\quad}$$

- $\{A, B\}$

- $\{0, 1\}$

- A

- B

- ?

$$A \times 0 \rightarrow \textcircled{A}$$

$$A \times 0 \rightarrow \textcircled{B}$$

$$A \times 1 \rightarrow \textcircled{A}$$

$$B \times 0 \rightarrow \phi$$

$$B \times 1 \rightarrow \phi$$

$$A^1 \rightarrow A, B, AB, \phi$$

$$(Q, \Sigma, q_0, F, \delta)$$

Q = Set of all states

Σ = inputs

q_0 = start state / initial state

F = Set of final states

$$\delta = Q \times \Sigma \rightarrow \underline{2^Q}$$

- $\{A, B\}$

- $\{0, 1\}$

- A

- B

- ?

$$A \times 0 \rightarrow \textcircled{A}$$

$$A \times 0 \rightarrow \textcircled{B}$$

$$A \times 1 \rightarrow \textcircled{A}$$

$$B \times 0 \rightarrow \phi$$

$$B \times 1 \rightarrow \phi$$

$$A^1 \rightarrow A, B, AB, \phi - 2^2 - 4$$

3 States - A, B, C

$$A^1 \rightarrow A, B, C, AB, AC, BC, ABC, \phi$$

$$2^3 - 8$$

Implementing an NFA in Code:

You can handle nondeterminism using three methods:

1. **Recursive backtracking algorithm:** Cycle through all alternatives and make recursive calls until you find a path reaching a final state.
2. **State vector:** Maintain a set or vector keeping track of every possible state the NFA occupies simultaneously.
3. **Quantum computing:** Theoretically, a quantum computer explores all possibilities simultaneously.

4. Pushdown Automata (PDA)

A DFA cannot recognize context-free languages because a DFA cannot count indefinitely. A DFA cannot process the language $\{a^n b^n \mid n \geq 0\}$ or a language of palindromes ww . These require a stack.

Pushdown Automata (Introduction)

A Pushdown Automata (PDA) is a way to implement a Context Free Grammar in a similar way we design Finite Automata for Regular Grammar

- > It is more powerful than FSM
- > FSM has a very limited memory but PDA has more memory
- > PDA = Finite State Machine + A Stack

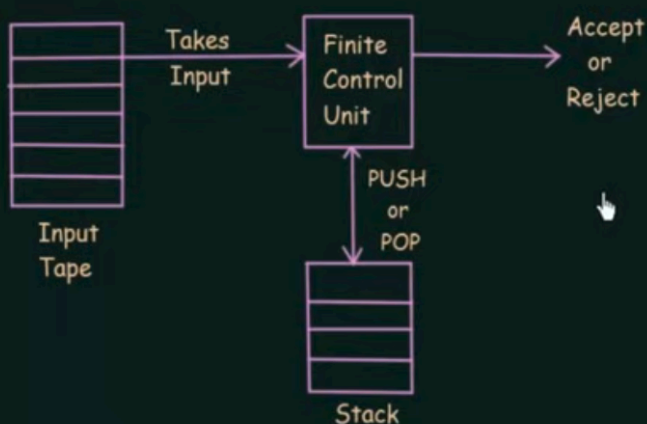
A PDA is an NFA with a stack attached. It parses context-free languages, whose structure resembles many modern programming languages. Compilers rely on PDAs for essential architectural features.

PDA Operation

A PDA reads an infinitely long tape from left to right. When it reads a blank cell marked with λ , it halts. It cannot reverse directions or write to the tape. (<https://youtu.be/eY7fwj5jvC4?si=251yb7yzXpndtFUV>)

A Pushdown Automata has 3 components:

- 1) An input tape
- 2) A Finite Control Unit
- 3) A Stack with infinite size



Pushdown Automata (Formal Definition)

A Pushdown Automata is formally defined by 7 Tuples as shown below:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

where,

Q = A finite set of States

Σ = A finite set of Input Symbols

Γ = A finite Stack Alphabet

δ = The Transition Function

q_0 = The Start State

z_0 = The Start Stack Symbol

F = The set of Final / Accepting States

δ takes as argument a triple $\delta(q, a, X)$ where:

(i) q is a State in Q

(ii) a is either an Input Symbol in Σ or $a = \epsilon$

(iii) X is a Stack Symbol, that is a member of Γ

δ = The Transition Function

q_0 = The Start State

z_0 = The Start Stack Symbol

F = The set of Final / Accepting States

δ takes as argument a triple $\delta(q, a, X)$ where:

(i) q is a State in Q

(ii) a is either an Input Symbol in Σ or $a = \epsilon$

(iii) X is a Stack Symbol, that is a member of Γ

The output of δ is finite set of pairs (p, γ) where:

p is a new state

γ is a string of stack symbols that replaces X at the top of the stack

Eg. If $\gamma = \epsilon$ then the stack is popped

If $\gamma = X$ then the stack is unchanged

If $\gamma = YZ$ then X is replaced by Z and Y is pushed onto the stack

The stack provides infinite storage functioning as Last-In, First-Out memory. Processing begins with an empty stack. The machine typically pushes a bottom-of-the-stack marker (like ϵ) first. The PDA uses two alphabets: one for tape characters and one for stack characters.

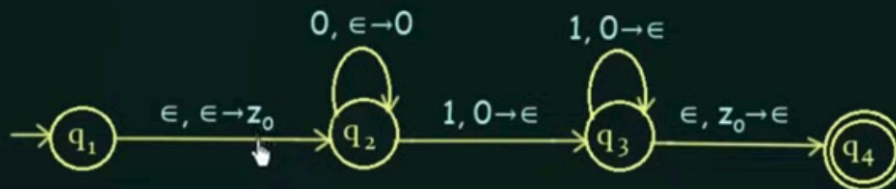
A transition checks the current state, reads the tape cell, and pops the top of the stack. It then moves to a new state and pushes characters onto the stack. If a transition specifies for the tape read, the stack pop, or the string push, it means the machine skips that exact action.

Example PDA Trace for $\{a^n b^n \mid n \geq 0\}$:

1. Start in state 0. Move to state 1 and push ϵ to the stack.

2. While reading a from the tape, pop the top character, then push it back followed by an $.$ You count the a symbols by pushing $.$ for every a you read.
3. When you see a b , change to state 2 and pop an $.$. Continue popping one $.$ for every b read.
4. If the string is formatted correctly, you will read a $.$ from the tape exactly when you pop the $.$ from the stack. The machine moves to state 3, an accept state.

Example: Construct a PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$



\$

0011 ✓

Types of PDAs

1. **Deterministic PDA (DPDA):** Every input string has exactly one unique path.

Pushdown Automata - Example (Even Palindrome) PART-1

Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$

PALINDROMES: A word or sequence that reads the same backwards as forwards.

Examples: NOON

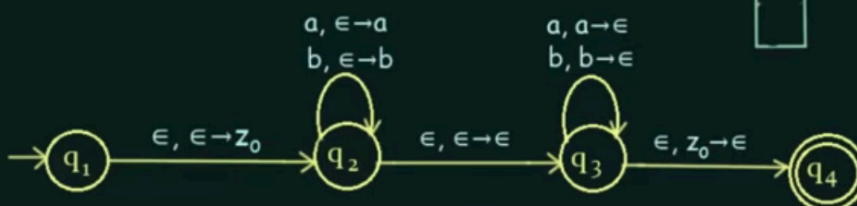
NO LEMON NO MELON

123321

abba

RACECAR

a b b a ✓



2. **Nondeterministic PDA (NPDA):** The machine chooses among several paths. A string is accepted if at least one path succeeds.

Unlike finite automata, an NPDA is strictly more powerful than a DPDA. There are specific languages an NPDA accepts that a DPDA cannot.

5. Formal Grammars

A grammar describes languages. In English, you use rules to construct meaningful sentences. You define a rule like `<sentence> -> <noun-phrase> <predicate>`. You then define `<noun-phrase> -> <article> <noun>` and `<predicate> -> <verb>`. If you set `<article>` to {a, an, the}, `<noun>` to {cow, bird, boy}, and `<verb>` to {eats, runs}, you can generate sentences like "the boy eats."

A formal grammar generalizes this. It uses a precise mathematical system of symbols and production rules to define the exact syntax of valid strings. It does not define meaning. A formal grammar acts as a language generator and forms the backbone for parsers in compilers.

Formal Definition

A grammar G is defined as a quadruple $G = (\Sigma, \Sigma, P, S)$:

- (Non-terminals or variables): A finite, non-empty set of symbols that can be replaced.
- (Terminals or Σ): A finite, non-empty set of unchangeable symbols making up the final string.
- P (Production rules): A finite set of rules dictating how to replace symbols, formatted as $\alpha \rightarrow \beta$, where $\alpha \in (\Sigma^+)$ and $\beta \in (\Sigma^*)$.
- S (Start Symbol): A specific non-terminal in Σ where all derivations begin.

Derivations

Production rules transform strings. Given a string α and a rule $\alpha \rightarrow \beta$, you rewrite α to obtain β . You denote this derivation as $\alpha \Rightarrow \beta$. You apply rules in arbitrary order as many times as needed to derive a final string containing only terminals.

Example 1:

Let $G = (\Sigma, \Sigma, P, S)$, where $\Sigma = \{S\}$, $\Sigma = \{a, b\}$, and $P = \{S \rightarrow ab, S \rightarrow aSb\}$.

- $S \rightarrow ab$
- $S \rightarrow aSb \rightarrow aabb$
- $S \rightarrow aSb \rightarrow aaSbb \rightarrow aaabbbb$ This grammar generates the language $L(G) = \{a^i b^i \mid i \geq 1\}$. You get a string of the form $a^n b^n$.

Example 2:

Let $V = \{S, \}$, $\Sigma = \{a, b, c\}$, S is start, and $P = \{S \rightarrow aSc, S \rightarrow abc, a \rightarrow a, b \rightarrow bb\}$.

This grammar defines the language $L(G) = \{a^n b^n c^n \mid n \geq 1\}$. The string contains one or more a 's, followed by the same number of b 's, followed by the same number of c 's. For $n = 3$, the derivation yields $a^3 b^3 c^3$.

The Chomsky Hierarchy

Noam Chomsky classified generative grammars in 1956 based on the strictness of their production rules.

1. **Type 0 (Unrestricted):** The rule form is $A \rightarrow \alpha$. There are no restrictions on A or α . These are sentential forms. This grammar is parsed by Turing machines. It is generally too broad for computer languages.
2. **Type 1 (Context-Sensitive):** The rule form is $A \rightarrow \alpha B \beta$. A is a single non-empty non-terminal. The production depends entirely on the surrounding context symbols. Example: $S \rightarrow a, b/c, \dots$. Linear bounded automata parse these.
3. **Type 2 (Context-Free):** The rule form is $A \rightarrow \alpha$. The left-hand side is exactly one single non-terminal. Context does not matter. Most programming languages use context-free grammars. PDAs parse these. Example: $S \rightarrow aSb, S \rightarrow ab$ generates $\{a^n b^n\}$.
4. **Type 3 (Regular):** The rule form is $A \rightarrow a$ or $A \rightarrow aB$. The left-hand side is a single non-terminal. The right-hand side is restricted to an empty string, a single terminal, or a single terminal followed by a non-terminal. Finite state machines parse these in linear time. Lexical scanners use regular grammars. Example: rules $S \rightarrow a, a \rightarrow a, b \rightarrow b, \epsilon$ generate $\{a^n b^m \mid n, m \geq 1\}$.

6. Parsing

Parsing is the process of breaking down a complex stream of data or text into smaller, readable components based on a set of formal grammar rules. It converts raw, unstructured data into organized structures like trees or tables that humans or machines interpret and execute.

A language is a well-defined set of strings requiring a membership criterion. You use context-free grammars as that exact membership criterion. If you find a general algorithm using the grammar to recognize strings, you can parse the language. Parsing a string simply means finding a derivation or a derivation tree for that exact string.